Software Requirements

Minsoo Ryu

Hanyang University

Topics covered

- What are requirements?
- Classifications of Requirements
  - Functional and Non-Functional Requirements
  - User and System Requirements
- Requirements Specification Languages
- Guidelines for Requirements Specification

What Are Requirements?

- The requirements are the descriptions of the services provided by the system and its operational constraints

- Requirements engineering is the process of finding out, analyzing, documenting, and checking these services and constraints

Topics covered

- What is Requirements Engineering?
- Classifications of Requirements
  - Functional and Non-Functional Requirements
  - User and System Requirements
- Requirements Specification Languages
- Guidelines for Requirements Specification

Functional Requirements

- The functional requirements describe what the system should do
  - Depend on the type of software, expected users and the type of system where the software is used

- Examples
  - Authentication of a user when he/she tries to log into the system
  - System shutdown in the case of a cyber attack
  - Verification email is sent to user whenever he/she registers for the first time on some software system

Non-functional Requirements

- Non-functional requirements are requirements that are not directly concerned with the specific functions delivered by the system
  - They may relate to emergent system properties such as reliability, response time, and store occupancy
- Examples
  - Emails should be sent with a latency of no greater than 12 hours
  - Mail server is secure enough so that attackers cannot read the email messages being sent to and from your company
- Non-functional requirements may be more critical than functional requirements
  - If these are not met, the system is useless

Classifications of Non-functional Requirements

- **Product requirements**
  - Requirements which specify that the delivered product must behave in a particular way (execution speed, reliability, etc)
- **Organisational requirements**
  - Requirements which are a consequence of organisational policies and procedures (process standards used, implementation requirements, etc)
- **External requirements**
  - Requirements which arise from factors which are external to the system and its development process (interoperability requirements, privacy requirements, ethical requirements, etc)

Functional and Non-functional Requirements

- In reality, the distinction between different types of requirements is not as clear-cut as the simple definitions suggest
  - A user requirement concerned with security may appear to be a non-functional requirement
  - However, when developed in more detail, this requirement may generate other requirements that are clearly functional such as the need to include user authentication facilities in the system

A Problem with Non-functional Requirements

- A common problem with non-functional requirements is that they can be difficult to verify
  - Users or customers often state these requirements as general goals such as ease of use, the ability of the system to recover from failure or rapid user response

- Whenever possible, you should write non-functional requirements quantitatively so that they can be objectively tested

User and System Requirements

- By Davis:
  - If a company wishes to let a contract for a large software development project, the company must define its needs in a sufficiently abstract way that a solution is not predefined
  - Once a contract has been awarded, the contractor must write a system definition for the client in more detail so that the client understands and can validate what the software will do

- Example (user requirements)
  - A government agency, which manages huge numbers of documents and other records, wishes to move from a paper based approach to a system that can manage digital documents

User Requirements

- User requirements should describe functional and non-functional requirements in such a way that they are understandable by users who don't have detailed technical knowledge
  - They should only specify the external behaviour of the system and should avoid, as far as possible, system design characteristics
  - You should not use software jargon, structured notations or formal notations, or describe the requirement by describing the system implementation
  - User requirements are defined using natural language, tables and diagrams as these can be understood by all users

System Requirements

- System requirements are expanded versions of the user requirements
  - They may be used as part of the contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system
- System requirements should not be concerned with how the system should be designed or implemented
  - However, at the level of detail required to completely specify a complex software system, it is impossible, in practice, to exclude all design information

Topics covered

- What is Requirements Engineering?
- Classifications of Requirements
  - Functional and Non-Functional Requirements
  - User and System Requirements
- Requirements Specification Languages
- Guidelines for Requirements Specification

Requirements Specification Languages

- Natural language-based specification
- Form-based specification
- Tabular specification
- Graphical specification
- Formal specification

Natural Language-based Specification

- Natural language is often used to write system requirements specification
  - However, because system requirements are more detailed than user requirements, natural language specifications can be confusing and hard to understand

- System requirements are often written with more specialized notations
  - Structured natural language
  - Design description languages
  - Graphical notation
  - Mathematical specifications

Form-based Specification

- When a standard form is used for requirements specification, the following information should be included
  - Definition of the function or entity
  - Description of inputs and where they come from
  - Description of outputs and where they go to
  - Indication of other entities required
  - Pre and post conditions (if appropriate)
  - The side effects (if any) of the function

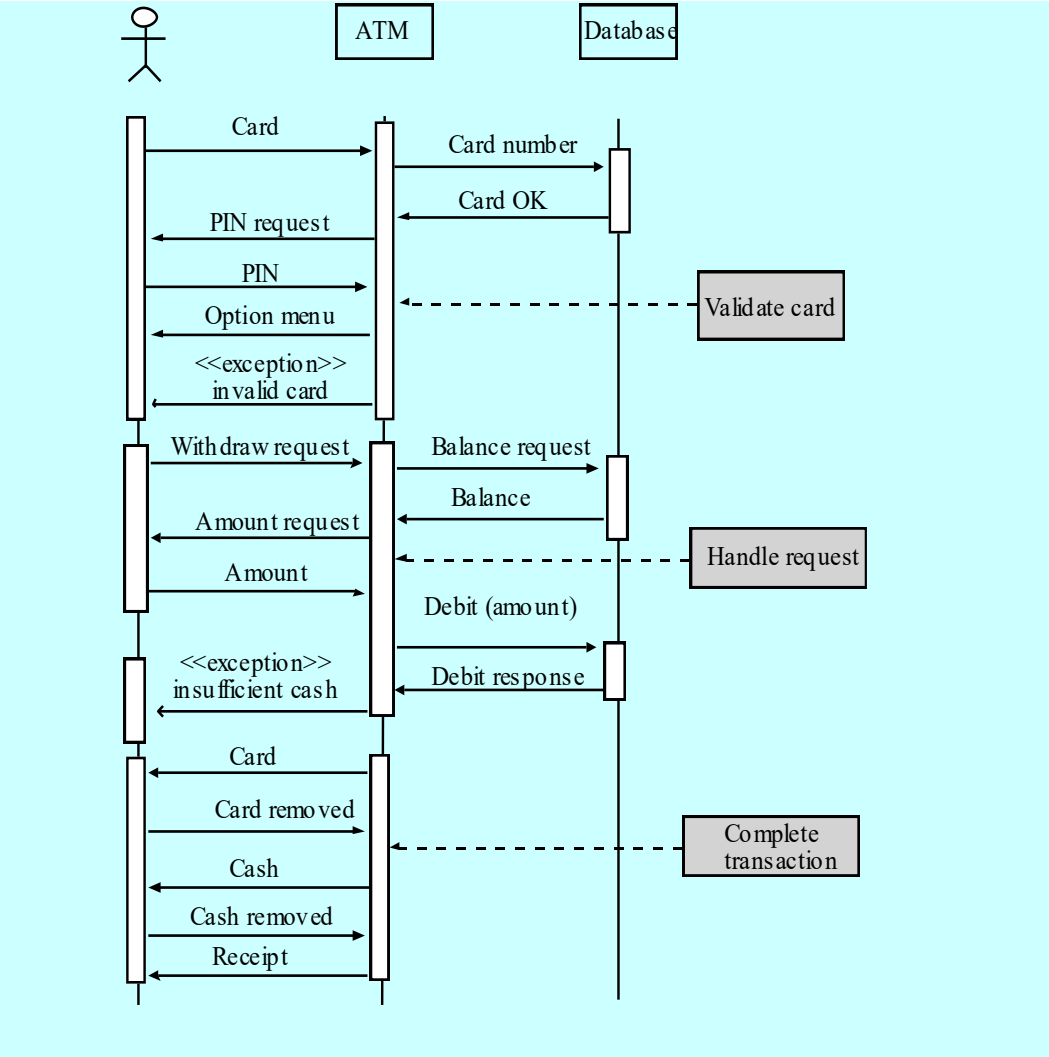
Form-based Specification (Insulin Pump)

Insulin Pump/Control Software/SRS/3.3.2	
Function	Compute insulin dose: Safe sugar level
Description	Computes the dose of insulin to be delivered when the current measured sugar level is in the safe zone between 3 and 7 units.
Inputs	Current sugar reading (r2), the previous two readings (r0 and r1)
Source	Current sugar reading from sensor. Other readings from memory.
Outputs	CompDose Š the dose in insulin to be delivered
Destination	Main control loop
Action:	CompDose is zero if the sugar level is stable or falling or if the level is increasing but the rate of increase is decreasing. If the level is increasing and the rate of increase is increasing, then CompDose is computed by dividing the difference between the current sugar level and the previous level by 4 and rounding the result. If the result, is rounded to zero then CompDose is set to the minimum dose that can be delivered.
Requires	Two previous readings so that the rate of change of sugar level can be computed.
Pre-condition	The insulin reservoir contains at least the maximum allowed single dose of insulin..
Post-condition	r0 is replaced by r1 then r1 is replaced by r2
Side-effects	None

Tabular Specification

Condition	Action
Sugar level falling (r2 < r1)	CompDose = 0
Sugar level stable (r2 = r1)	CompDose = 0
Sugar level increasing and rate of increase decreasing ((r2-r1)<(r1-r0))	CompDose = 0
Sugar level increasing and rate of increase stable or increasing. ((r2-r1) □ (r1-r0))	CompDose = round ((r2-r1)/4) If rounded result = 0 then CompDose = MinimumDose

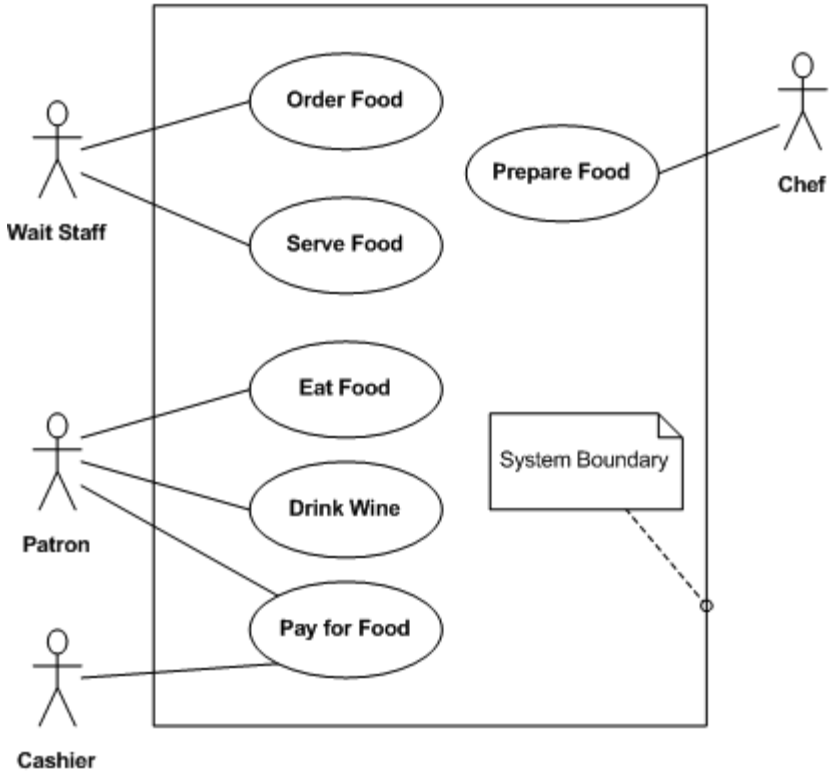
Graphical Specification (ATM withdrawal)



Visual Specification Languages

- **UML (Unified Modeling Language)**
  - **Visual and object-oriented**
  - **Diagrams + natural languages**

Use Case Diagram



Formal Specification Languages

- **The Vienna Development Method (VDM) is one of the longest-established Formal Methods for the development of computer-based systems**
  - **Originating in work done at IBM's Vienna Laboratory in the 1970s, it has grown to include a group of techniques and tools based on a formal specification language - the VDM Specification Language (VDM-SL)**

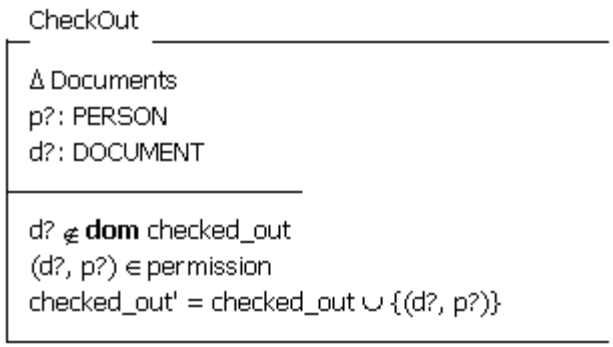
- **Features like programming languages**
  - **Various types: numeric, character, token and quote types**
    - Type constructors are used to build more structured data types
  - **Collections and operators: sets, mappings, and sequences**

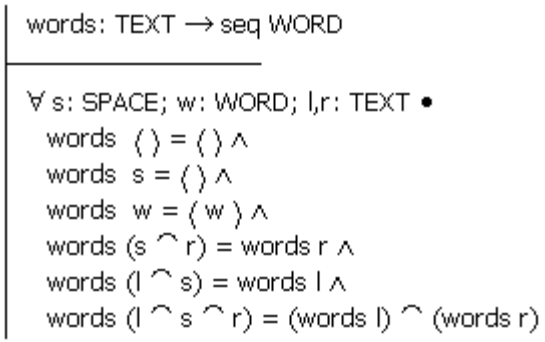
Formal Specification Languages

- Example 1:
  - SQRT(x:nat)r:real
  - post r*r = n and r>=0
- Example 2:
  - SQRTP(x:real)r:real
  - pre x >=0
  - post r*r = n and r>=0

Formal Specification Languages

- The Z notation (formally pronounced /zed/), named after Zermelo-Fränkel set theory, is a formal specification language used for describing and modeling computing systems





Topics covered

- What is Requirements Engineering?
- Classifications of Requirements
  - Functional and Non-Functional Requirements
  - User and System Requirements
- Requirements Specification Languages
- Guidelines for Requirements Specification

Problems with Natural Language

- **Lack of clarity**
  - It is sometimes difficult to use language in a precise and unambiguous way without making the document wordy and difficult to read
- **Requirements confusion**
  - Functional requirements, non-functional requirements, system goals, and design information may not be clearly distinguished
- **Requirements amalgamation**
  - Several different requirements may be expressed together

Unambiguity

- **Imprecision in the requirements specification is the cause of many software engineering problems**
  - **Ambiguous requirements may be interpreted in different ways by developers and users**

- **Consider the term 'appropriate viewers'**
  - **Customer intention - special purpose viewer for each different document type**
  - **Developer interpretation - Provide a text viewer that shows the contents of the document**
    - The requirement is worded ambiguously
    - A developer might claim that the requirement had been met

Completeness and Consistency

- **In principle, requirements should be both complete and consistent**

- **Complete**
  - **They should include descriptions of all facilities required**
  - **MECE (mutually exclusive, collectively exhaustive)**
    - No overlaps, no gaps

- **Consistent**
  - **There should be no conflicts or contradictions in the descriptions of the system facilities**

Testability

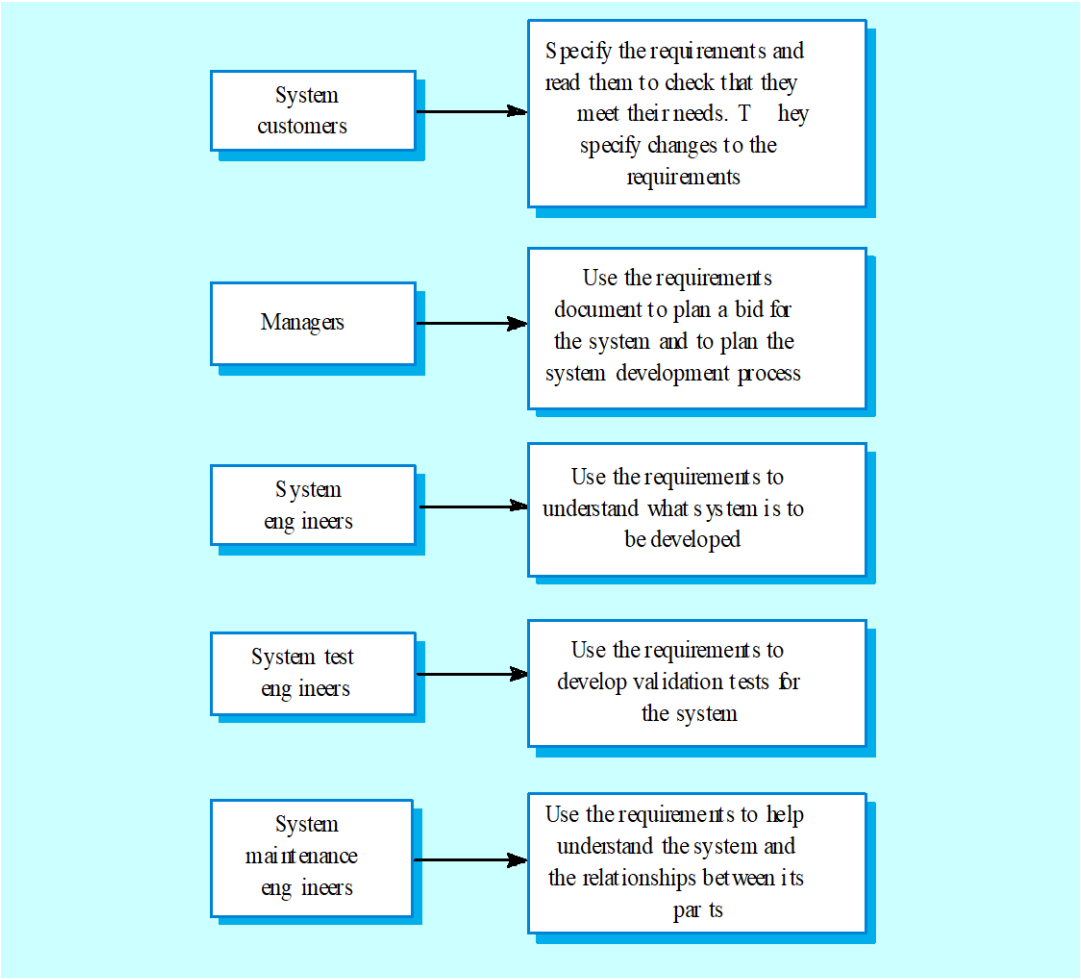
- **Most requirements should be testable**
  - If this is not the case, another verification method should be used instead (e.g. analysis, inspection or review of design)
  - Testable requirements are an important component of validation
- **Certain requirements, by their very structure, are not testable**
  - These include requirements that say the system shall never or always exhibit a particular property
  - Proper testing would require an infinite testing cycle
- **Un-testable non-functional requirements may still be kept as a documentation of customer intent**
  - However they are usually traced to process requirements that are determined to be a practical way of meeting them
  - For example, a non-functional requirement to be free from backdoors may be satisfied by replacing it with a process requirement to use pair programming

The Requirements Document

- **The requirements document is the official statement of what is required of the system developers.**
  - **Should include both a definition of user requirements and a specification of the system requirements**
  - **It is NOT a design document**
    - As far as possible, it should set of WHAT the system should do rather than HOW it should do it

- **The requirements document has a diverse set of users**
  - **The diversity of possible users means that the requirements document has to be a compromise between communicating the requirements to customers, defining the requirements in precise detail for developers and testers, and including information about possible system evolution**

Users of a Requirements Document



IEEE Requirements Standard (IEEE Std. 830-1998)

- **Defines a generic structure for a requirements document that must be instantiated for each specific system.**
  - **Introduction.**
    - Purpose of the requirements document
    - Scope of the product
    - Definitions, acronyms, and abbreviations
    - References
    - Overview of the remainder of the document
  - **General description.**
    - Product perspective
    - Product functions
    - User characteristics
    - General constraints
    - Assumptions and dependencies
  - **Specific requirements.**
    - Functional, non-functional, and interface requirements
    - (standard structure has not been defined)
  - **Appendices.**
  - **Index.**


Q & A

요구사항에 대한 고찰

- 요구사항은 소프트웨어라는 문제에 대한 정의**(PROBLEM DEFINITION)**이다
  - 제대로 된 요구사항을 얻지 못한다면, 그 이후에 프로젝트를 얼마나 잘 수행하느냐는 논할 가치도 없다
- 완벽한 요구사항이란 없다
  - 소프트웨어 요구사항들은 “그때 그때 다르다”는 말로 쉽게 표현할 수 있을 정도로 불확실성이 크다
  - 빠진 요구사항이 없다는 것을 확인할 방법은 없으나, 특정 사항은 기록할 필요성을 느끼지 못해 빼먹는 경우는 있다
- **70%** 가량의 프로젝트 일정 지연을 프로젝트 시작 전에 요구사항 정의를 명확하게 함으로써 피할 수 있다
- 요구사항은 모호할 지 몰라도 제품은 명확할 것이다
  - 제품이 완성되면 요구사항은 확정된다. 중요한 것은 구현된 요구사항이 고객을 만족시킬 수 있느냐라는 점이다.